Lab 02 — Standard Errors and Randomization Tests

AUTHOR
Ashish Pant

PUBLISHED
May 26, 2026

Load libraries

Load tidyverse library

```
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✔ dplyr     1.2.1     ✔ readr     2.2.0
✔ forcats   1.0.1     ✔ stringr   1.6.0
✔ ggplot2   4.0.3     ✔ tibble    3.3.1
✔ lubridate 1.9.5     ✔ tidyr     1.3.2
✔ purrr     1.2.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```
library(here) # resolves files from the project root regardless of where qmd lives
```

```
here() starts at /mnt/data1/documents/ISU/RProjects/STAT5101
```

Writing functions in R

Load data (user header=TRUE)

```
creativity <- read.csv(here("data/creativity.csv"), header = TRUE)
head(creativity)
```

```
  treatment score
1 intrinsic  12.0
2 intrinsic  12.0
3 intrinsic  12.9
4 intrinsic  13.6
5 intrinsic  16.6
6 intrinsic  17.2
```

Define function for computing the standard error

```
se <- function(x) {
    s <- sd(x, na.rm = TRUE)
    n <- sum(!is.na(x))
    s / sqrt(n)
}
```

Calculate standard error for all 47 observation scores (just the score column)

Calculate standard error for all 47 observations (just the score column)

```
se(creativity$score)
```

```
[1] 0.7653046
```

Using tidyverse, summarize the standard error (a different way to calculate)

```
creativity |>
    summarize(
        se = se(score)
    )
```

```
         se
1 0.7653046
```

Group by treatment and compute the standard error by treatment group

```
creativity |>
    group_by(treatment) |>
    summarize(
        se = se(score)
    )
```

```
# A tibble: 2 × 2
  treatment    se
  <chr>     <dbl>
1 extrinsic 1.10
2 intrinsic 0.906
```

Randomization Tests

First set seed for reproducibility

```
set.seed(5101)
```

Calculate the observed difference in mean score (observed means as documented in creativity.csv)

```
observed_diff <- creativity |>
    group_by(treatment) |>
    summarize(mean_score = mean(score)) |>
    summarize(diff = diff(mean_score)) |>
    pull(diff)

observed_diff
```

```
[1] 4.100725
```

Run randomization (basically, simulate what else, other than what was recorded in the csv, could have been observed at random)

```
nsim <- 10000
```

```
nsim <- 10000
simulated_diffs <- replicate(nsim, {
    # Everthing in {} is evaluated as if it was a function (this is like a lambda)
    creativity |>
        # generate a random treatment value
        mutate(shuffled = sample(treatment)) |>
        # group by the random treatment value
        group_by(shuffled) |>
        # Calculate the mean and diff like before
        summarize(mean_score = mean(score)) |>
        summarize(diff = diff(mean_score)) |>
        # pull the difference
        pull(diff)
})
head(simulated_diffs)
```

```
[1]  2.7043478  1.4356884  0.9077899 -1.6465580  0.8141304  1.8614130
```

Calculate the p-value

```
p_value <- (1 + sum(abs(simulated_diffs) >= abs(observed_diff))) / (nsim + 1)
p_value
```

```
[1] 0.00489951
```

Explanation of p-value calculation: The p-value is the number of observations with a value equal to or more extreme (>=) than the observed value divided by the number of observations. This would translate to:

sum(abs(simulated_diffs) >= abs(simulated_diff)) / nsim

But, this results in a 0 p-value if by chance the observed_diff was an extreme value itself and none of the simulated values were equal or larger resulting in 0 / nsim = 0, which is an impossible p-value.

By adding 1 to both the numerator and denominator, essentially, we are including the observed_diff in the calculation. Since observed_diff is equal to itself, it is valid to include it as a count in the numerator and increment the denominator by 1 as if we performed 10001 simulations instead of 10000.

Self Assessment

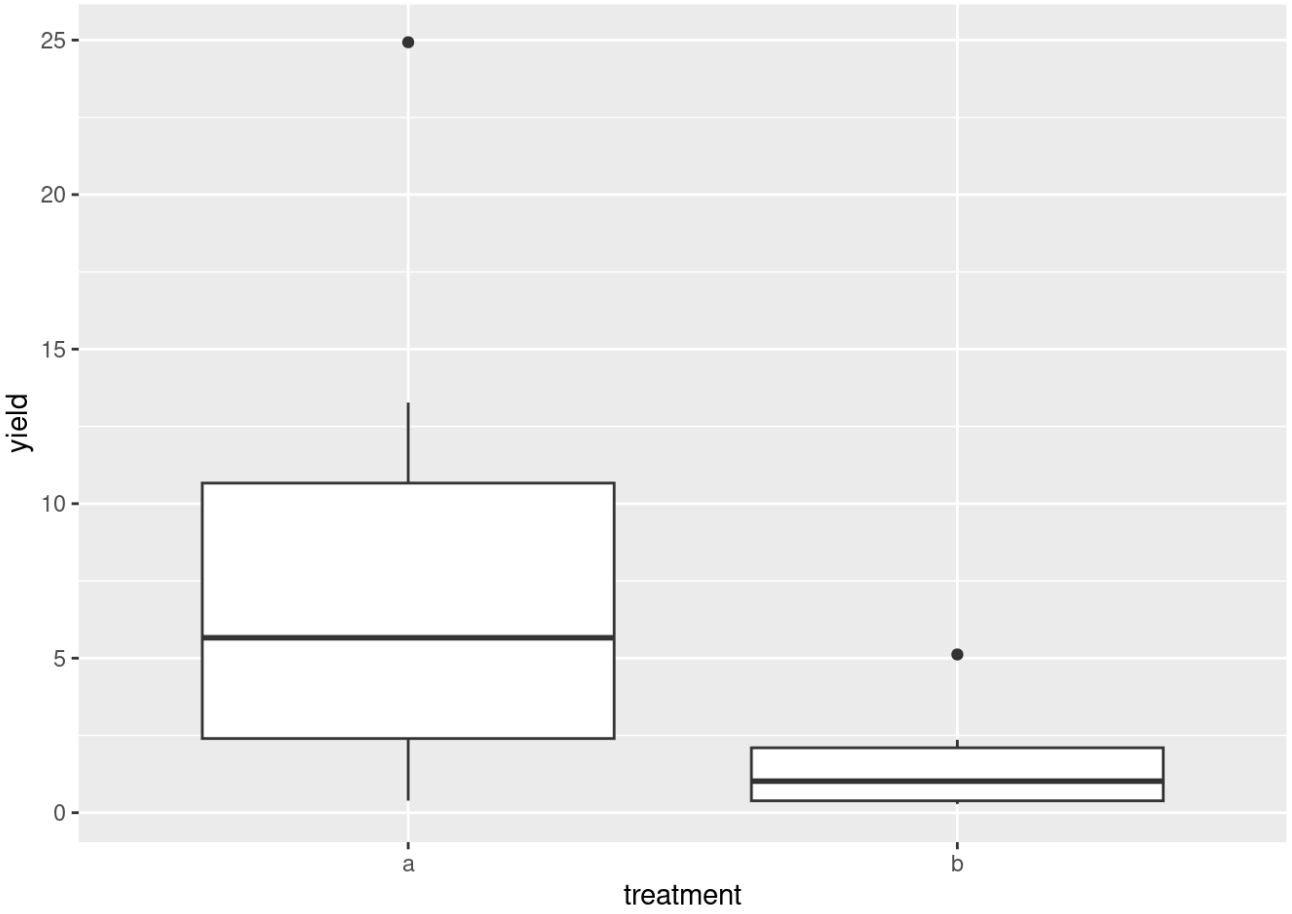
Read in the tomato data, tomato.csv

```
tomato <- read.csv(here("data/tomato.csv"), header = TRUE)
head(tomato)
```

```
  treatment yield
1         a  0.39
2         b  0.44
3         a  1.83
4         a  5.66
5         b  0.33
6         a  8.06
```

(a) Draw side-by-side box plots of the yield for each fertilizer

```
ggplot(data = tomato, mapping=aes(x = treatment, y = yield)) +
    geom_boxplot()
```



(b) Estimate the difference in mean yield between the two fertilizers (a-b)

```
observed_diff = tomato |>
    group_by(treatment) |>
    summarize(mean_yield = mean(yield)) |>
    summarize(diff = diff(mean_yield)) |>
    pull(diff)
observed_diff
```

```
[1] -6.531429
```

(c) Estimate the difference in median yield

```
tomato |>
    group_by(treatment) |>
    summarize(median_yield = median(yield)) |>
    summarize(diff = diff(median_yield)) |>
    pull(diff)
```

```
[1] -4.64
```

(d) Use a randomization test to test the null hypothesis of no effect of the fertilizer on mean yield. Report the two-sided p-value

Run randomization test and collect simulated diffs

```
# Set seed for reproducibility
set.seed(5101)

# We have already calculated the observed diff above
# Run randomization test

nsim <- 10000
simulated_diffs <- replicate(nsim, {
    tomato |>
        mutate(shuffled = sample(treatment)) |>
        group_by(shuffled) |>
        summarize(mean_yield = mean(yield)) |>
        summarize(diff = diff(mean_yield)) |>
        pull(diff)
})
head(simulated_diffs)
```

```
[1]  2.305714 -1.914286  4.940000  2.337143 -1.040000 -3.597143
```

Calculate p-value

```
p_value = (1 + sum(abs(simulated_diffs) >= abs(observed_diff))) / (nsim + 1)
p_value
```

```
[1] 0.03659634
```

Comments: The observed difference in in mean yields is 6.53. The p-value of 0.037 is less than 0.05 but greater than 0.01 which weakly rejects the null hypothesis that the treatment has no effect.